



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Búsqueda Lineal y Prevención de Desbordamiento
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumno participante:	Chacha Chango Victor Manuel
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Utilizar estructuras de datos unidimensionales (arreglos) y algoritmos de recorrido para la gestión y búsqueda de información en memoria.

Específicos:

- Implementar el algoritmo de búsqueda lineal sobre arreglos paralelos para correlacionar datos (productos y precios).
- Recorrer estructuras de datos iterables aplicando sentencias de control de flujo como break para optimizar el rendimiento.
- Prevenir errores de desbordamiento de memoria (Buffer Overflow) mediante la validación estricta de los límites en los ciclos iterativos.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará en forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++.
- Subir en un documento en formato pdf que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código generado, y subirlo al aula virtual.
- Si se manifiestan dudas se invita a los estudiantes a acudir a las tutorías académicas.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- - Computador o laptop.
- - Plataforma educativa de la UTA.



- Internet para consultas.
- - Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas

2.7 Resultados obtenidos

EJERCICIO 1: BÚSQUEDA LINEAL DE PRODUCTOS

Enunciado

Desarrollar un programa en C++ o Java que permita buscar un producto dentro de un arreglo utilizando el algoritmo de búsqueda lineal.

Indicaciones: Crear un arreglo con 5 nombres de productos. Crear otro arreglo paralelo con sus precios. Solicitar al usuario el nombre del producto a buscar. Recorrer el arreglo usando búsqueda lineal. Mostrar: Nombre del producto, Precio correspondiente. Utilizar break cuando se encuentre el producto. Mostrar un mensaje si el producto no existe.

Análisis del problema

Se requiere la inicialización de dos arreglos paralelos de tamaño 5: uno de tipo cadena (String) para almacenar los nombres de los productos y otro de tipo real (Double/Float) para los precios correspondientes. El usuario introducirá una cadena de texto a buscar. Mediante un ciclo repetitivo Para, se comparará la cadena ingresada con cada elemento del arreglo de nombres. Si hay coincidencia, se imprimirá el nombre y el precio accediendo al mismo índice i del arreglo paralelo, se activará una bandera booleana (encontrado = verdadero) y se usará la sentencia break para salir del ciclo inmediatamente y no gastar recursos. Si el ciclo termina y la bandera sigue siendo falsa, se mostrará un mensaje de que el producto no existe.

Variable de entrada, salida y proceso

Entrada

productoBuscar -> (Cadena)

Proceso

nombres[5] -> (Arreglo de Cadenas)

precios[5] -> (Arreglo de Reales)

encontrado -> (Booleano)

i -> (Entero iterador)

Salida

Impresión del nombre del producto y su precio, o mensaje de error.



Algoritmos (PSeInt)

Inicio

Definir arreglos nombres y precios de tamaño 5.

Definir variables productoBuscar (cadena), encontrado (lógico) y el iterador i.

Asignar 5 productos al arreglo nombres y sus 5 precios correspondientes al arreglo precios.

Inicializar la variable encontrado en Falso.

Pedir al usuario que ingrese el nombre del producto a buscar y leer productoBuscar.

Iniciar un ciclo Para desde i = 0 hasta 4.

Dentro del ciclo, Si nombres[i] es igual a productoBuscar Entonces:

Imprimir el nombre del producto y el precios[i].

Cambiar encontrado a Verdadero.

Forzar la salida del ciclo (asignando i <- 4 en PSeInt para emular el break).

FinPara

Si encontrado es Falso Entonces:

Imprimir "El producto no existe en el inventario".

Fin

Pseudocódigo (PSeInt)

Algoritmo BusquedaProductos

 Dimension nombres[5]

 Dimension precios[5]

 Definir nombres, productoBuscar Como Cadena

 Definir precios Como Real

 Definir encontrado Como Logico

 Definir i Como Entero

 nombres[0] <- "Teclado"; precios[0] <- 15.50

 nombres[1] <- "Mouse"; precios[1] <- 8.75

 nombres[2] <- "Monitor"; precios[2] <- 120.00

 nombres[3] <- "Auriculares"; precios[3] <- 25.00

 nombres[4] <- "Microfono"; precios[4] <- 45.20

 encontrado <- Falso

 Escribir "Ingrese el nombre del producto a buscar: "

 Leer productoBuscar

 Para i <- 0 Hasta 4 Con Paso 1 Hacer

 Si nombres[i] = productoBuscar Entonces

 Escribir "Producto encontrado: ", nombres[i]

 Escribir "Precio correspondiente: \$", precios[i]

 encontrado <- Verdadero

 i <- 4

 FinSi

FinPara

Si encontrado = Falso Entonces

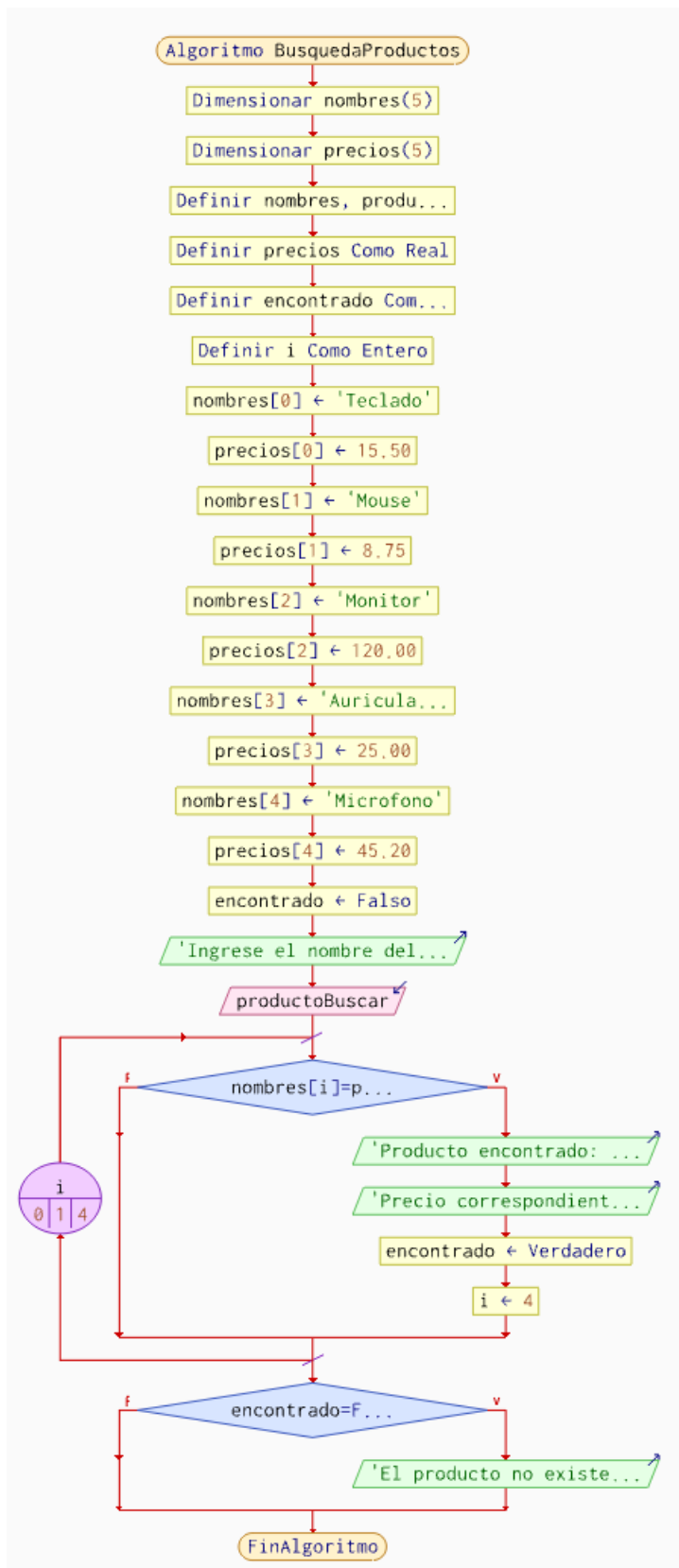
 Escribir "El producto no existe en el inventario."

FinSi

FinAlgoritmo



Diagramas de Flujo





Código en C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombres[5] = {"Teclado", "Mouse", "Monitor", "Auriculares", "Microfono"};
    double precios[5] = {15.50, 8.75, 120.00, 25.00, 45.20};
    string productoBuscar;
    bool encontrado = false;

    cout << "Ingrese el nombre del producto a buscar: ";
    cin >> productoBuscar;

    for (int i = 0; i < 5; i++) {
        if (nombres[i] == productoBuscar) {
            cout << "\nProducto encontrado: " << nombres[i] << endl;
            cout << "Precio correspondiente: $" << precios[i] << endl;
            encontrado = true;
            break;
        }
    }

    if (!encontrado) {
        cout << "\nEl producto no existe en el inventario." << endl;
    }

    return 0;
}
```

Código en Java

```
import java.util.Scanner;

public class BusquedaProductos {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String[] nombres = {"Teclado", "Mouse", "Monitor", "Auriculares",
"Microfono"};
        double[] precios = {15.50, 8.75, 120.00, 25.00, 45.20};
        boolean encontrado = false;

        System.out.print("Ingrese el nombre del producto a buscar: ");
        String productoBuscar = scanner.nextLine();

        for (int i = 0; i < nombres.length; i++) {
            if (nombres[i].equals(productoBuscar)) {
                System.out.println("\nProducto encontrado: " + nombres[i]);
            }
        }
    }
}
```



```
System.out.println("Precio correspondiente: $" + precios[i]);
encontrado = true;
break;
    }
}

if (!encontrado) {
    System.out.println("\nEl producto no existe en el inventario.");
}

scanner.close();
}
```

3 casos de prueba de escritorio

Caso de Prueba	Variables de Entrada (productoBuscar)	Condición (nombres[i] == productoBuscar)	Acción (break y bandera)	Salida del Sistema
1. Producto en la primera posición	"Teclado"	Verdadero en i=0	encontrado = true, ejecuta break.	Muestra "Producto encontrado: Teclado", "Precio: \$15.5".
2. Producto intermedio	"Auriculares"	Falso en 0, 1, 2. Verdadero en i=3.	encontrado = true, ejecuta break.	Muestra "Producto encontrado: Auriculares", "Precio: \$25".
3. Producto inexistente	"Laptop"	Falso en todos los índices.	Nunca cambia la bandera.	Muestra "El producto no existe en el inventario."



EJERCICIO 2: PREVENCIÓN DE BUFFER OVERFLOW

Enunciado

Realizar un programa en C++ o Java que muestre los elementos de un arreglo sin provocar desbordamiento de memoria.

Indicaciones: Crear un arreglo con 6 números enteros. Utilizar un ciclo for para recorrer el arreglo. Validar correctamente los límites del ciclo. Mostrar cada posición y su valor. Agregar un comentario explicando qué ocurriría si el ciclo supera el tamaño del arreglo.

Análisis del problema

El problema radica en inicializar un arreglo de longitud fija (6 elementos). En C++ los arreglos están indexados desde el 0, lo que significa que un arreglo de tamaño 6 tiene como índices válidos del 0 al 5. Se usará un ciclo for que inicie en 0 y su condición de continuidad será estrictamente menor que 6 ($i < 6$). El programa debe imprimir de forma concatenada el índice actual y el valor alojado en dicho índice. Finalmente, se debe justificar teóricamente mediante comentarios dentro del código las consecuencias de un "Buffer Overflow" (ejemplo: leer en la posición 6 de un arreglo de 6 espacios).

Variable de entrada, salida y proceso

Entrada

Ninguna externa

Proceso

numeros[6] -> (Arreglo de enteros)

i -> (Entero iterador)

Salida

Impresión de la posición del índice i y el valor numeros[i]

Algoritmos (PSeInt)

Inicio

Definir arreglo numeros de tamaño 6.

Definir variable i como entero.

Asignar 6 valores enteros fijos a las posiciones del arreglo numeros (del 0 al 5).

Mostrar en pantalla un mensaje de inicio de recorrido.

Iniciar un ciclo Para con la variable i desde 0 hasta 5 con paso 1.

Dentro del ciclo, mostrar la frase "Posición [", el valor de i, "] : Valor = " y el contenido de numeros[i].

FinPara

Fin

Pseudocódigo (PSeInt)

Algoritmo PrevencionBufferOverflow

 Dimension numeros[6]

 Definir numeros, i Como Entero

 numeros[0] <- 10

 numeros[1] <- 20

 numeros[2] <- 30

 numeros[3] <- 40

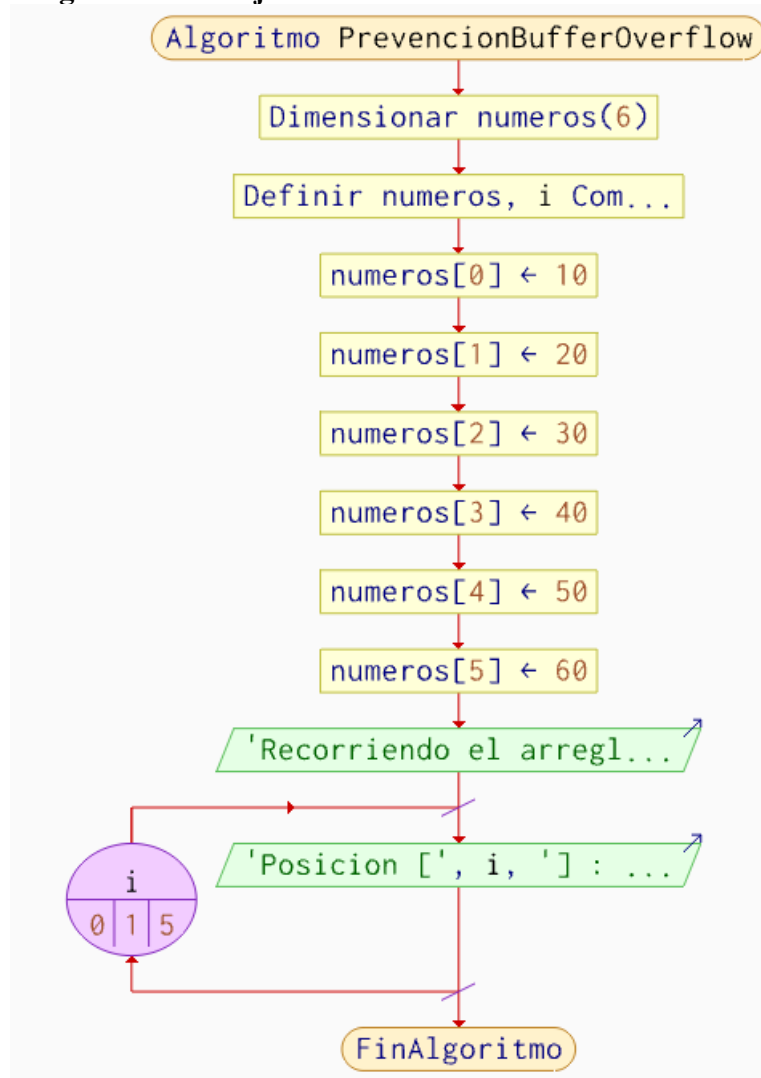
 numeros[4] <- 50

 numeros[5] <- 60



Escribir "Recorriendo el arreglo de forma segura:"
Para i <- 0 Hasta 5 Con Paso 1 Hacer
 Escribir "Posicion [", i, "]" : Valor = ", numeros[i]
FinPara
FinAlgoritmo

Diagramas de Flujo



Código en C++

```
#include <iostream>
using namespace std;

int main() {
    int numeros[6] = {10, 20, 30, 40, 50, 60};

    cout << "=== MOSTRANDO ELEMENTOS DEL ARREGLO ===" << endl;

    for (int i = 0; i < 6; i++) {
        cout << "Posicion [" << i << "] -> Valor: " << numeros[i] << endl;
    }
}
```




```
return 0;  
}
```

Código en Java

```
public class PrevencionBufferOverflow {  
    public static void main(String[] args) {  
        int[] numeros = {10, 20, 30, 40, 50, 60};  
  
        System.out.println("=== MOSTRANDO ELEMENTOS DEL ARREGLO ===");  
  
        for (int i = 0; i < 6; i++) {  
            System.out.println("Posicion [" + i + "] -> Valor: " + numeros[i]);  
        }  
    }  
}
```

3 casos de prueba de escritorio

Iteración	Condición (i < 6)	Ejecución y Salida	Estado de memoria
Inicio (i = 0)	Verdadero	Muestra: "Posicion [0] - > Valor: 10"	Acceso seguro a la dirección base.
Mitad (i = 3)	Verdadero	Muestra: "Posicion [3] - > Valor: 40"	Acceso seguro (base + 3 posiciones).
Límite y cierre (i = 5 a i = 6)	Verdadero en 5, Falso en 6	Muestra: "Posicion [5] - > Valor: 60". Luego se rompe el ciclo.	Acceso seguro al último índice; el límite evita el overflow.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

- La implementación del algoritmo de búsqueda lineal utilizando arreglos paralelos demostró ser una técnica muy efectiva para almacenar y correlacionar información



relacionada, permitiendo vincular exitosamente el nombre de un producto con su precio correspondiente en memoria.

- El uso de sentencias de control de flujo, específicamente la instrucción break, resultó fundamental para optimizar el rendimiento del algoritmo; al interrumpir el ciclo iterativo tan pronto como se encuentra la coincidencia, se evita el consumo innecesario de recursos computacionales.
- Se evidenció que la validación estricta de los límites en los ciclos iterativos (asegurando, por ejemplo, que el iterador sea estrictamente menor a la longitud del arreglo, como $i < 6$) es esencial para prevenir el desbordamiento de memoria (Buffer Overflow) y garantizar la ejecución segura y estable del programa.

2.10 Recomendaciones

- Se recomienda el uso constante de variables booleanas (como la bandera encontrada utilizada en el ejercicio) en los algoritmos de búsqueda, ya que facilitan el control del flujo del programa y permiten disparar de forma controlada los mensajes de error cuando un dato no está en el inventario.
- Al programar en lenguajes como C++ o Java, es aconsejable prestar especial atención a la declaración de los límites en los bucles for; al iniciar los índices siempre en 0, el límite superior debe ser estrictamente menor al tamaño total para evitar lecturas de memoria fuera del arreglo.
- Es altamente recomendable diseñar e implementar casos de prueba de escritorio que abarquen distintos escenarios (buscar el primer elemento, un elemento intermedio, y un dato inexistente) para validar la lógica y asegurar que las prevenciones contra errores funcionen antes de la fase de codificación.

2.11 Referencias bibliográficas

2.12 Anexos

Link de Github

[https://github.com/VictorChacha1/VictorChacha1-SUBTEMA-4 Operaciones-Avanzadas-e-Integridad-de-Datos Victor-Chacha.git](https://github.com/VictorChacha1/VictorChacha1-SUBTEMA-4_Operaciones-Avanzadas-e-Integridad-de-Datos_Victor-Chacha.git)

Evidencias del Ejercicio 1

En C++:

```
Ingrese el nombre del producto a buscar: Teclado
Producto encontrado: Teclado
Precio correspondiente: $15.5
```

```
Ingrese el nombre del producto a buscar: Microfono
Producto encontrado: Microfono
Precio correspondiente: $45.2
```

En java:

```
Ingrese el nombre del producto a buscar: Mouse
Producto encontrado: Mouse
Precio correspondiente: $8.75
```



Ingrese el nombre del producto a buscar: Monitor

Producto encontrado: Monitor

Precio correspondiente: \$120.0

Evidencias del Ejercicio 2

En C++:

```
=== MOSTRANDO ELEMENTOS DEL ARREGLO ===  
Posicion [0] -> Valor: 10  
Posicion [1] -> Valor: 20  
Posicion [2] -> Valor: 30  
Posicion [3] -> Valor: 40  
Posicion [4] -> Valor: 50  
Posicion [5] -> Valor: 60
```

En java:

```
=== MOSTRANDO ELEMENTOS DEL ARREGLO ===  
Posicion [0] -> Valor: 10  
Posicion [1] -> Valor: 20  
Posicion [2] -> Valor: 30  
Posicion [3] -> Valor: 40  
Posicion [4] -> Valor: 50  
Posicion [5] -> Valor: 60
```